B.Sc. (ECS)-I

Unit-1

In Previous year we learned basic concepts of Python like tokens, control structure, lists, tuples etc.

Now in this this course we are going to learn some advanced concepts of Python programming.

Unit 1 contains Advanced conditional and looping statements and Regular expression, let's understand them one by one.

Advanced Conditional and Looping Statements

Before moving towards our point, first we have to revise looping in short.

Looping(Loops):

A Loop is sequence of instruction/s that repeated until certain condition is reached.

There are following loops present in python

i) for loop

ii) while loop

i) for loop:

- The for loop in Python is used to iterate over a sequence (list,tuple,string) or other iterable objects.
- Iterating over a sequence is called traversal.

Syntax of for Loop

```
for variablename in sequence:
            Body or logic
```

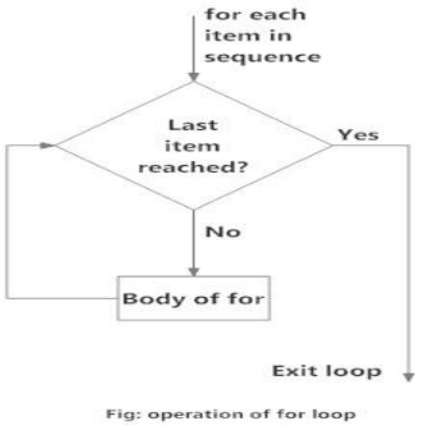


Fig: operation of for loop

Here each item from sequence is assigned to variable one by one. The loop terminates when there

Is no any item left in sequence.

Program of for loop

```
x=[0,1,5]
for p in x:
    print(p)
```

In our program **x** is a List(sequence) which have three Elements.

By using **for** loop each element of list is assigned to Variable **p** after that these elements printed by Function **print()**

for loop with range() function:

We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).

We can also define the start, stop and step size **as range(start, stop-1,step_size-1)**. step_size defaults to 1 if not provided

Program

```
for p in range(1,10):
    print(p)
```

We used range() function to create a sequence of numbers as,

Range(1,10) where 1 is start and 10 is stop. By this a sequence if **1** to (10-1)=**9** is created.

The step size is optional part.

ii) while loop :

- The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.
- We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax of while Loop in Python

```
while test_expression:
          Body of while
```

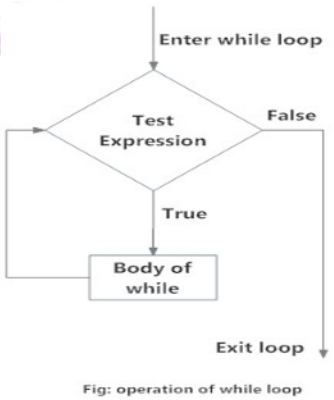


Fig: operation of while loop

Program to print 1 to 10 using while loop

```
i=1
while(i<=10):
    print(i)
    i+=1
```

Now let's see Advanced conditional and looping statements.

In Python we can flat lists for that many techniques are used.

Among these techniques one is the List Comprehension

List:

A list is a collection of elements or items which is ordered and changeable.

List allows duplicate members.

In Python lists are written with square brackets.

List Comprehension:

List comprehension is a concept where we can create new lists based on existing iterables (like lists, tuples, or strings) in short and efficient manner.
It provides a more compact syntax compared to traditional for loops for similar operations.

Example without List Comprehension

```
marks=[10,20,30,40,50]
new=[]
for x in marks:
            new.append(x+5)
print(new)
```

Output: [15, 25, 35, 45, 55]

Example with List Comprehension

```
marks=[10,20,30,40,50]
new=[x+5 for x in marks]
print(new)
print(type(new))
```

Output: [15, 25, 35, 45, 55]
<class 'list'>

Tuple:

A tuple is a collection of elements which is ordered and unchangeable.
Tuple allows duplicate members.In Python tuples are written with round brackets.
Python does not support direct tuple comprehension because a tuple is an immutable.
We can achieve tuple comprehension by type casting.

Tuple Comprehension:

Tuple comprehension is a concept where we can create new Tuple based on existing iterables (like lists, tuples, or strings) in short and efficient manner.
It provides a more compact syntax compared to traditional for loops for similar operations.

Example without Tuple Comprehension

```
marks=(10,20,30,40,50)
new=()
for x in marks:
        new=new+(x+5,)
print(new)
```

Output: (15, 25, 35, 45, 55)

Example with Tuple Comprehension

```
marks=(10,20,30,40,50)
new=tuple(x+5 for x in marks)
print(new)
print(type(new))
```

Output: (15, 25, 35, 45, 55)
<class ' tuple'>

Set:

Set is a collection of elements which is unordered and indexed.
There are no duplicate members in Set.
In Python set is written with curly brackets.

Set Comprehension:

Set comprehension is a concept where we can create new Set based on existing iterables (like lists, tuples, or strings) in short and efficient manner.
It provides a more compact syntax compared to traditional for loops for similar operations.
Example without Set Comprehension

```
marks={10,20,30,40,50}
t=list(marks)
p=[]
for x in t:
    p.append(x+5)
new=set(p)
print(new)
```

Output: {35, 45, 15, 55, 25}

Example with Tuple Comprehension

```
marks=(10,20,30,40,50)
new=set({x+5 for x in marks})
print(new)
print(type(new))
```

Output: {35, 45, 15, 55, 25}
<class ' set'>

Dictionary:

A dictionary is a built-in data structure that stores data in key-value pairs. It is an ordered, changeable collection of unique key-value pairs.

Dictionary Comprehension:

Dictionary comprehension is a concept where we can create new dictionary based on existing iterables (like lists, tuples, or strings) in short and efficient manner.
It provides a more compact syntax compared to traditional for loops for similar operations.

Example without dictionary Comprehension

```
num=[1,2,3,4,5]
new={}
for n in num:
    new[n]=n**2
print(new)
print(type(new))
```

Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
<class 'dict'>

Example with dictionary Comprehension

```
num=[1,2,3,4,5]
new={n:n**2 for n in num}
print(new)
print(type(new))
```

Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
<class 'dict'>

Use of global, local and nonlocal in functions:

We know a variable is name given to memory location where data is stored.
There are three scopes of variable as,

a) global b) Local c)nonlocal

a) global scope/global variable:

In Python, a variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.

```
x = 100      #global variable
def show():
    print("x inside:", x)
show()  #call to show function
print("x outside:", x)
```

Output: x inside: 100
x outside: 100

b) local scope/local variable:

A variable which is declared inside the function's body or in the local scope is known as a local variable. A Local Variable can not be accessed outside the local scope

```
def show():
    x = 10 #local variable
    print("x inside:", x)

show()  #call to show function
#print("x outside:", x) //error
```

Output: x inside: 10

c) nonlocal scope/nonlocal variable:

In Python, the nonlocal keyword is used within nested functions to indicate that a variable is not local to the inner function, but rather belongs to an enclosing function's scope.

This allows you to modify a variable from the outer function within the nested function, while still keeping it distinct from global variables.

```
def show():
    x = 10
    print("x in show:", x)
    def inner():
        nonlocal x
        x=20
        print("x in inner:", x)
    inner()#call to in function
show()  #call to show function
```

Output:
x in show: 10
x in inner 20

Anonymous/Lambda Function : In Python, an anonymous function is a function that is defined without a name. While normal functions are defined using the def keyword in Python, anonymous functions are defined using the lambda keyword.
Hence, anonymous functions are also called lambda functions.
Syntax of Lambda Function in python

Syntax

lambda arguments: expression

```
double = lambda x: x * 2
     print(double(5))
```

Output: 10

Higher Order Function :

A higher-order function is a function that can take other functions as arguments and/or return functions as outputs. These functions allow us to write more brief, flexible, and abstract code.

```
def add(a,b):
    return(a+b)

def display(func):      #higher order function
    return(func(10,20))

print(display(add))
```

Output:30

Type Hinting/Type Hint/Type Annotation:

In static-typed languages like Java or C++, we declare the datatype of variables before using them.
Python is a dynamically typed language means there is no need to declare data types manually.
In python data type is assigned to a variable at run time.
Python has support for optional "type hints".
These **"type hints"** or annotations are a special syntax that allow declaring the type of a variable.
This helps to catch errors before runtime.

```
def add(x: int, y: int) -> int:       #type hint
    return x + y

a:int=20  #type hint
b:int=30  #type hint
print(add(a,b)) #call
```

Output-50

filter() in Python:

We can use the filter() function in Python to extract elements from a sequence that meet a certain condition.
Instead of writing a full loop with if statements, the Python filter() function allows express filtering logic in a clear way.

Syntax: filter(function, iterable)

```
# A list of numbers
x = [10, 19, 21, 30, 15]

# Define a function that returns True for adults
def find(age):
    return age >= 18

# Use filter() to get only above 17
y = filter(find, x)

# Convert the iterator to a list to view it
p = list(y)
print(p)
```

Output: [19, 21, 30]

map() in Python:

map() is a built-in function that applies a given function to each item of a sequence and returns a map object.

The map() function takes minimum two arguments: one is function to apply and second is a sequence, e.g. a list, string, or tuple.

Here we can pass multiple sequences as arguments.

Syntax: map(function, sequence)

```
def plus(x):
    return x + 5

num=[1, 2, 3, 4, 5]
result = map(plus, num)
print(list(result))
```

Output: [6, 7, 8, 9, 10]

Another example for map function to add two lists

```
a=[1,2]
b=[3,4]

result = map(lambda x,y: x + y, a,b)
print(list(result))
```

Output:[4,6]

reduce() in Python:

The reduce(fun,seq) function is used to apply a particular function passed in its argument to all of the list elements mentioned in the sequence passed along.

The reduce() function applies cumulatively to the elements of sequence.

```
from functools import reduce

# Function to add two numbers
def add(x, y):
    return x + y

a = [1, 2, 3, 4, 5]
res = reduce(add, a)
print(res)
```

Output: 15

The reduce() function applies add() cumulatively to the elements in numbers. First, 1 + 2 = 3. Then, 3 + 3 = 6. And so on, until all numbers are processed.
The final result is 15.

Using reduce() with lambda

```
from functools import reduce

# Summing numbers with reduce and lambda
a = [1, 2, 3, 4, 5]
res = reduce(lambda x, y: x + y, a)

print(res)
```

Output: 15

Recursive functions/Function Recursion :

Python recursion is a technique where a function calls itself to solve a problem in a step-by-step manner. It is commonly used for solving problems that can be broken down into smaller subproblems, such as calculating factorials, Fibonacci sequences etc.

```
def countdown(n):
    if n <= 0:  # Base case
        print("Done!")
    else:
        print(n)
        countdown(n - 1)  # Recursive call

countdown(3)
```

Iterables:

An iterable is any Python object which return its members one at a time, permitting it to be iterated over in a for-loop.
Examples of iterables are lists, tuples, strings, set, dictionary.

```
x=[0,1,5]    #x is an iterable
for p in x:
    print(p)
```

Iterator:

Iterator is an object that allows sequential traversal through elements in a collection(lists, tuples, dictionaries, sets)

Iterators are memory-efficient because they fetch elements one at a time rather than loading an entire sequence into memory.

Iterator is an object which produces the next item in a sequence each time when we call next() on it.

We get an iterator by calling the built-in iter() function on an iterable.

The for loop in Python uses iterators automatically behind the scenes.

```
x=[0,1,5]  #x is an iterable
it=iter(x) #creating iterator
print(next(it))
print(next(it))
print(next(it))
```

Output: 0
1
5

Example of iterators in for loop

```
x=[0,1,5]    #x is an iterable
for p in iter(x):
    print(p)
```

Output: 0
1
5

Custom Iteration with Classes

We can create custom iterators by defining a class that implements __iter__() and __next__().

This is useful when iterating over data structures that require special processing.

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1

counter = Counter(1, 5)
for num in counter:
    print(num)
```

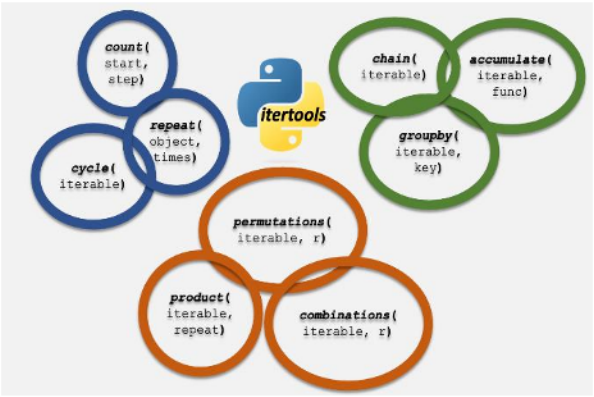
Output:1 2 3 4 5

itertools functions:

Itertools functions are divided into categories as

1) Infinite Iterators **2) Terminating Iterators**(Iterators Terminating on the Shortest Input Sequence)

3) Combinatoric Iterators



1) Infinite Iterators

Sometimes iterators can be infinite. Such type of iterators are known as Infinite iterators. Python provides three types of infinite iterators -

a) count() b) cycle() c) repeat()

a) count (start, step): This iterator starts printing from the "start" number and prints infinitely. If steps are mentioned, the numbers are skipped else step is 1 by default. See the below example for its use with for in loop.

```
import itertools
for i in itertools.count(5, 5):
    #if i == 35:
    #    break
    #else:
        print(i, end =" ")
```

b) cycle(iterable): This iterator prints all values in order from the passed container. It restarts printing from beginning again when all elements are printed in a cyclic manner.

```
import itertools
count = 0
# for in loop
for i in itertools.cycle('AB'):
    #if count > 7:
    #    break
    #else:
        print(i, end = " ")
        count += 1
```

c) **repeat(val, num):** This iterator repeatedly prints the passed value infinite number of times. If the optional keyword num is mentioned, then it repeatedly prints num number of times.

```
import itertools

# using repeat() to repeatedly print number
print ("Printing the numbers repeatedly : ")
print (list(itertools.repeat(25, 4)))
```

2) Terminating Iterators(Iterators Terminating on the Shortest Input Sequence):
Functions that return a single iterable after using up all elements in the input iterable are called terminating iterators. They are used to reduce the input iterable in some way.
Python provides three types of terminating iterators -
a) accumulate() b) groupby() c) chain()

a) accumulate(iterable, func):
The accumulate() function provides a way to get the sum of values or the sum of the outcomes of other binary operations. **In the absence of a specified function, addition will be used.**
The output is always in the form of list
Syntax:

```
itertools.accumulate(iterable, func=None)
```

where,
-iterable: The input sequence (like a list, tuple, or set).
-func (optional): A binary function (e.g., operator.add, operator.mul, max, etc.). **If not provided, addition is used by default.**
Examples:
itertools.accumulate() without binary function

```
import itertools

a = [1, 2, 3, 4]
print(list(itertools.accumulate(a)))
```

Output: [1, 3, 6, 10]

Using itertools.accumulate() with **operator.mul**

```
import itertools
import operator

a = [1, 2, 3, 4, 5]
res = itertools.accumulate(a, operator.mul)
print(list(res))
```

Output:[1, 2, 6, 24, 120]

Using itertools.accumulate() with **max**

```
import itertools

a = [5, 3, 6, 2, 1, 9, 1]
res = itertools.accumulate(a, max)
print(list(res))
```

Output: [5, 5, 6, 6, 6, 9, 9]

```
# accumulate([5], func=max) -> 5
# accumulate([5, 3], func=max) -> 5
# accumulate([5, 3, 6], func=max) -> 6
# accumulate([5, 3, 6, 2], func=max) -> 6
# accumulate([5, 3, 6, 2, 1], func=max) -> 6
# accumulate([5, 3, 6, 2, 1, 9], func=max) -> 9
# accumulate([5, 3, 6, 2, 1, 9, 1], func=max) -> 9
```

b) groupby(iterable, key):
The groupby() function is used for grouping consecutive elements of an iterable that have the same value. It generates a group of consecutive elements from an iterable and **returns them containing a key and a group.**

```
import itertools

L = [("a", 1), ("a", 2), ("b", 3), ("b", 4)]

key_func = lambda x: x[0]

for key, group in itertools.groupby(L, key_func):
      print(key + " :", list(group))
```

Output:
a : [('a', 1), ('a', 2)]
b : [('b', 3), ('b', 4)]

itertools.groupby() groups the list of tuples based on the first element of each tuple. The key function (key_func) extracts the first element of each tuple (x[0]). The groupby() function then categorizes the tuples into groups with the same first element, and each group is displayed in the output.

c) chain(iterables):
The chain() function is used to treat multiple sequences as one continuous sequence.

```
from itertools import chain

list_1 = ["A", "B"]
list_2 = [1, 2, 3]
s = "san"

for p in chain(list_1, list_2, s):
    print(p)
```

Output:
A
B
1
2
3
s
a
n

chain.from_iterable(iterable):
This function comes under the category of terminating iterators. This function takes a single iterable as an argument and returns a flattened iterable containing all the elements of the input iterable.

```
from itertools import chain
x = chain.from_iterable(['sangola','Mahavidyalaya'])
# printing the flattened iterable
print(list(x))
```

Output: ['s', 'a', 'n', 'g', 'o', 'l', 'a', 'M', 'a', 'h', 'a', 'v', 'i', 'd', 'y', 'a', 'l', 'a', 'y', 'a']

batched(iterable, n):
This function is used to create an iterator that returns batches of a specified size from an iterable. This function is useful for processing data in fixed-size chunks.
If the number of elements in the iterable is not a multiple of the batch size, the final batch will contain the remaining elements.

```
from itertools import batched
data = list(range(6))
# Create batches of size 3
for batch in batched(data, 3):
        print(batch)print(list(x))
```

Output: (1, 2, 3)
(4, 5)

compress(data, selectors):
This function is used to filter elements from an iterable based on a corresponding selector iterable. It returns only those elements for which the corresponding value in the selector is True.

```
import itertools

data = [1, 2, 3, 4, 5]
selectors = [1, 0, 1, 0, 1]  # 1 means include, 0 means exclude
filtered = itertools.compress(data, selectors)
for item in filtered:
     print(item)
```

Output:
1
3
5

dropwhile(predicate, iterable):
This function is used to drop elements from an iterable as long as a specified condition is True. Once the condition becomes False, the function returns the remaining elements without checking further.

```
import itertools

def show(x):
   return x < 5

data = [1, 12, 3, 5, 6, 7]
result = itertools.dropwhile(show, data)
for num in result:
     print(num)
```

Output: 12
3
5
6
7
Here, we drop numbers from the list while they are less than 5

filterfalse(predicate, iterable):
This function is used to filter elements from an iterable by removing those that satisfy a given predicate function. It returns only those elements for which the predicate evaluates to False.

```
import itertools

def is_even(n):
   return n % 2 == 0

numbers = [1, 2, 3, 4, 5, 6, 7, 8]
result = itertools.filterfalse(is_even, numbers)
for num in result:
   print(num)
```

Output:
1
3
5
7

islice(iterable, start, stop[, step]):
This function is used to return selected elements from an iterable.
This function is useful when working with large datasets or infinite iterators. By specifying a start, stop, and step, we can extract a portion of an iterable efficiently.

```
import itertools

numbers = range(20)
selected = itertools.islice(numbers, 0, 7, 3)
for num in selected:
      print(num)
```

Output: 0
3
6

pairwise(iterable):
This function is used to create an iterator that returns consecutive overlapping pairs from an iterable. It is useful for analyzing sequential relationships between elements in a dataset.

```
import itertools

numbers = [1, 2, 3, 4, 5]
pairs = itertools.pairwise(numbers)
for pair in pairs:
   print(pair)
```

Output:
(1, 2)
(2, 3)
(3, 4)
(4, 5)

```
import itertools

temperatures = [72, 75, 78, 74, 70, 69]
differences = [(b - a) for a, b in itertools.pairwise(temperatures)]
print(differences)
```

Output: [3, 3, -4, -4, -1]

starmap(function, iterable):
This function is used to apply a given function to arguments taken from an iterable of tuples. It is similar to the built-in map() function but instead of taking separate arguments, it expands each tuple as function arguments.

```
import itertools

def add(a, b):
   return a + b

pairs = [(2, 3), (4, 5)]
result = itertools.starmap(add, pairs)
for value in result:
   print(value)
```

Output:
5
9

takewhile(predicate, iterable):
This function is used to return elements from an iterable as long as a specified condition remains true. Once the condition evaluates to false, iteration stops immediately.
This function is useful for processing sequences where elements should only be included until a certain threshold is met.

```
import itertools

numbers = [1, 4, 12, 7, 15]
result = itertools.takewhile(lambda x: x < 10, numbers)
for num in result:
   print(num)
```

Output:
1
4

tee(iterable):

This function is used to create multiple independent iterators from a single iterable. These iterators share the same data but can be iterated over separately.

```
import itertools

numbers = [1, 2, 3]
result = itertools.tee(numbers,2)
for num in result:
   print(list(num))
```

Output:
[1,2,3]
[1,2,3]

zip_longest(iterable1, iterable2, fillval):

This function prints the values of iterables alternatively in sequence. If one of the iterables is printed fully, the remaining values are filled by the values assigned to fillvalue parameter.

```
import itertools
print ("The combined values of iterables is  : ")
print (*(itertools.zip_longest('sang', 'ola', fillvalue ='_' )))
```

Output: ('s', 'o') ('a', 'l') ('n', 'a') ('g', '_')

3) Combinatoric Iterators:

Combinatoric iterators are used to create different types of iterators that generate all possible combinations, permutations, or Cartesian products (a set of all ordered pairs) of an iterable.

Python provides three types of combinatoric iterators -

a) product() b) permutations() c) combinations()

a) product(iterable, repeat):

The product() function computes the Cartesian product of the input iterable. This is equivalent to nested for-loops. The repeat argument specifies the number of repetitions of the iterable. The result is the Cartesian product of the input iterable with itself, repeated the specified number of times.

```
from itertools import product
for item in product([1,2], [3,4]):
    print(item)
```

Output:(1, 3)
(1, 4)
(2, 3)
(2, 4)

b) permutations(iterable, r):

The permutations() function generates all possible permutations of the input iterable. We can specify the length of the permutations using the ‘r’ argument. If ‘r’ is not specified, then ‘r’ defaults to the length of the iterable.

```
from itertools import permutations
for item in permutations("AB", r=2):
    print(item)
```

Output:

('A', 'B')
('B', 'A')

c) combinations(iterable, r):

The combinations() function generates all possible combinations of the input iterable. The r argument specifies the length of the combinations. Unlike permutations, combinations don't consider the order of elements.

```
from itertools import combinations
for item in combinations(["A", "B", "C"], r=2):
                    print(item)
```

Output:

('A', 'B')
('A', 'C')
('B', 'C')

Combinations_with_replacement(iterable, r):

This function is used to generate all possible unique combinations of elements from a given iterable, allowing elements to be repeated in each combination.

This function is useful in combinatorial problems such as generating lottery numbers with repeated values, coin change problems, or rolling dice combinations.

```
import itertools
p = ['A', 'B']
result = itertools.combinations_with_replacement(p, 2)
for item in result:
        print(item)
```

Output:

('A', 'A')
('A', 'B')
('B', 'B')

Generators:

Generators allow us to create iterators in an efficient and memory-friendly way. Unlike regular functions, which return a single value and terminate, a generator can yield multiple values one at a time.

```
# Basic generator function
def count(n):
    count = 1
    while count <= n:
        yield count  # Yield the current count value
        count += 1

# Using the generator
counter = count(5)
for num in counter:
    print(num)
```

Output:

1
2
3
4
5

Generator Expression

Python also allows we to create generators using a generator expression, similar to list comprehensions but with parentheses.

```
squares = (x*x for x in range(1, 6))
print(list(squares))
```

Output: [1, 4, 9, 16, 25]

difference between Iterator vs Generators

Iterator	Generator
Class is used to implement an iterator	Function is used to implement a generator.
Local Variables aren't used here.	All the local variables before the yield function are stored.
Iterators are used mostly to iterate or convert other objects to an iterator using iter().	Generators are mostly used in loops to generate an iterator by returning all the values in the loop without affecting the iteration of the loop
Iterator uses iter() and next() functions	Generator uses yield keyword
Every iterator is not a generator	Every generator is an iterator

Closure:

Python closure is a nested function that allows us to access variables of the outer function even after the outer function is closed.

The main advantage of Python closures is that we can help avoid the using global values and provide some form of data hiding.

Closures allow inner functions to capture and retain the enclosing function's local state, even after the outer function has finished execution.

```
def fun1(x):

  # This is the outer function that takes an argument 'x'
    def fun2(y):

        # This is the inner function that takes an argument 'y'
        return x + y  # 'x' is captured from the outer function

    return fun2   # Returning the inner function as a closure

# Create a closure by calling outer_function
closure = fun1(10)

# Now, we can use the closure, which "remembers" the value of 'x' as 10
print(closure(5))
```

Decorators:

Decorators are a powerful and flexible way to modify or extend the behavior of functions or methods, without changing their actual code.
A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.

```
def make_pretty(func):
    # define the inner function
    def inner():
        # add some additional behavior to decorated function
        print("I got decorated")

        # call original function
        func()
    # return the inner function
    return inner

# define ordinary function
def ordinary():
    print("I am ordinary")

# decorate the ordinary function
decorated_func = make_pretty(ordinary)

# call the decorated function
decorated_func()
```

Output:

I got decorated

I am ordinary

In the example shown above, make_pretty() is a decorator. Notice the code,

```
decorated_func = make_pretty(ordinary)
```

We are now passing the ordinary() function as the argument to the make_pretty().
The make_pretty() function returns the inner function, and it is now assigned to the decorated_func variable.

```
decorated_func()
```

Here, we are actually calling the inner() function, where we are printing

@ Symbol With Decorator

Instead of assigning the function call to a variable, Python provides a much more simple way to achieve this functionality using the **@** symbol. For example,

```
def make_pretty(func):

    def inner():
        print("I got decorated")
        func()
    return inner
@make_pretty
def ordinary():
    print("I am ordinary")
ordinary()
```

Output:

I got decorated

I am ordinary

Getter and Setter in Python:

Getter: The getter method is used to retrieve the value of a private attribute. It allows controlled access to the attribute.

Setter: The setter method is used to set or modify the value of a private attribute. It allows to control how the value is updated, enabling validation or modification of the data before it's actually assigned.

```
class A:
     def __init__(self, age = 0):
          self._age = age
     # getter method
     def get_age(self):
          return self._age
     # setter method
     def set_age(self, x):
          self._age = x
x = A()
# setting the age using setter
x.set_age(21)
# retrieving age using getter
print(x.get_age())
print(x._age)
```

Using property() function

The property() method is used to wrap the getter, setter and deleter methods for an attribute, providing a more streamlined approach.

```
class A:
      def __init__(self, age = 0):
            self._age = age
      # getter method
      def get_age(self):
            return self._age
      # setter method
      def set_age(self, x):
            self._age = x
      age=property(get_age,set_age,del_age)
x = A()
x.age=21
print(x._age)
```

Using @property decorators

In this approach, the @property decorator is used for the getter and the @<property_name>.setter decorator is used for the setter. This approach allows a more elegant way to define getter and setter methods.

```
class A:
      def __init__(self):
            self._age = 0

      # using property decorator
      # a getter function
      @property
      def age(self):
            print("getter method called")
            return self._age

      # a setter function
      @age.setter
      def age(self, a):
            if(a < 18):
                  raise ValueError("age is below criteria")
            print("setter method called")
            self._age = a

x = A()
x.age = 21
print(x.age)
```

Magic methods:

Python Magic methods are the methods starting and ending with double underscores '__'. They are defined by built-in classes in Python and commonly used for operator overloading. They are also called Dunder methods, Dunder here means "Double Under (Underscores)". Built-in classes in Python define many magic methods. Use the print(dir(int)) function to see the number of magic methods

Initialization and Construction	Description
__new__(cls, other)	To get called in an object's instantiation.
__init__(self, other)	To get called by the __new__ method.
__del__(self)	Destructor method.
Unary operators and functions	**Description**
__pos__(self)	To get called for unary positive e.g. +someobject.
__neg__(self)	To get called for unary negative e.g. -someobject.
__abs__(self)	To get called by built-in abs() function.
__invert__(self)	To get called for inversion using the ~ operator.
__round__(self,n)	To get called by built-in round() function.
__floor__(self)	To get called by built-in math.floor() function.
__ceil__(self)	To get called by built-in math.ceil() function.
__trunc__(self)	To get called by built-in math.trunc() function.

Augmented Assignment	Description
__iadd__(self, other)	To get called on addition with assignment e.g. a +=b.
__isub__(self, other)	To get called on subtraction with assignment e.g. a -=b.
__imul__(self, other)	To get called on multiplication with assignment e.g. a *=b.
__ifloordiv__(self, other)	To get called on integer division with assignment e.g. a //=b.
__idiv__(self, other)	To get called on division with assignment e.g. a /=b.
__itruediv__(self, other)	To get called on true division with assignment
__imod__(self, other)	To get called on modulo with assignment e.g. a%=b.
__ipow__(self, other)	To get called on exponentswith assignment e.g. a **=b.
__ilshift__(self, other)	To get called on left bitwise shift with assignment e.g. a<<=b.
__irshift__(self, other)	To get called on right bitwise shift with assignment e.g. a >>=b.
__iand__(self, other)	To get called on bitwise AND with assignment e.g. a&=b.
__ior__(self, other)	To get called on bitwise OR with assignment e.g. a|=b.
__ixor__(self, other)	To get called on bitwise XOR with assignment e.g. a ^=b.
Type Conversion Magic Methods	**Description**
__int__(self)	To get called by built-int int() method to convert a type to an int.
__float__(self)	To get called by built-int float() method to convert a type to float.
__complex__(self)	To get called by built-int complex() method to convert a type to complex.
__oct__(self)	To get called by built-int oct() method to convert a type to octal.
__hex__(self)	To get called by built-int hex() method to convert a type to hexadecimal.

Augmented Assignment	Description
__index__(self)	To get called on type conversion to an int when the object is used in a slice expression.
__trunc__(self)	To get called from math.trunc() method.

String Magic Methods	Description
__str__(self)	To get called by built-int str() method to return a string representation of a type.
__repr__(self)	To get called by built-int repr() method to return a machine readable representation of a type.
__unicode__(self)	To get called by built-int unicode() method to return an unicode string of a type.
__format__(self, formatstr)	To get called by built-int string.format() method to return a new style of string.
__hash__(self)	To get called by built-int hash() method to return an integer.
__nonzero__(self)	To get called by built-int bool() method to return True or False.
__dir__(self)	To get called by built-int dir() method to return a list of attributes of a class.
__sizeof__(self)	To get called by built-int sys.getsizeof() method to return the size of an object.
Attribute Magic Methods	**Description**
__getattr__(self, name)	Is called when the accessing attribute of a class that does not exist.
__setattr__(self, name, value)	Is called when assigning a value to the attribute of a class.
__delattr__(self, name)	Is called when deleting an attribute of a class.

Operator Magic Methods	Description
__add__(self, other)	To get called on add operation using + operator
__sub__(self, other)	To get called on subtraction operation using - operator.
__mul__(self, other)	To get called on multiplication operation using * operator.
__floordiv__(self, other)	To get called on floor division operation using // operator.
__truediv__(self, other)	To get called on division operation using / operator.
__mod__(self, other)	To get called on modulo operation using % operator.
__pow__(self, other[, modulo])	To get called on calculating the power using ** operator.
__lt__(self, other)	To get called on comparison using < operator.
__le__(self, other)	To get called on comparison using <= operator.
__eq__(self, other)	To get called on comparison using == operator.
__ne__(self, other)	To get called on comparison using != operator.
__ge__(self, other)	To get called on comparison using >= operator.

Some examples of magic methods

1) __init__

```
class A:
    def __init__(self, x,y):
        self.x = x
        self.y = y
        print("Addition is",self.x+self.y)

b = A(21, 42)
```

Output: Addition is 63

2) __add__

```
num=10
res = num.__add__(5)
print(res)
```

Output:15

3) __str__

```
num=12
val = int.__str__(num)
print(type(val))
```

Output: <class 'str'>

Arbitrary Arguments *args : If we do not know how many arguments that will be passed into our function, add a * before the parameter name in the function definition.
This way the function will receive a tuple of arguments, and can access the items accordingly.

```
def fun(*x):
    print(x)
    print("\n element",x[2])

fun("Sangola","Solapur","Sangli")#call
```

Output : ('Sangola', 'Solapur', 'Sangli')
element sangli

Arbitrary Keyword Arguments, **kwargs :

If we do not know how many keyword arguments that will be passed into our function, add two asterisk: ** before the parameter name in the function definition.
This way the function will receive a dictionary of arguments, and can access the items accordingly:

```
def my_function(**kid):
    print("location is",kid["place"])

my_function(name="sachin",place="Mumbai")#call
```

Output: location is Mumbai

Garbage collection:

Garbage collection in Python is an **automated** memory management process that deletes objects that are not in use.

This process is helpful in memory management and minimizes the wastage of memory. The memory freed by garbage collection can be used for storing other important data or variables for the same program or by other programs.
Garbage collection in Python works automatically.
Also we can implement garbage collection manually by importing the **gc** module and using the **gc.collect()** function
Example of Automatic garbage collection

```
p=10
q=10


print(id(p))
print(id(q))

p=20
q=20

print(id(p))
print(id(q))
```

Example of manual garbage collection

```
import gc
p=10
q=10
print(id(p))
print(id(q))

p=20
q=20
gc.collect()
print(id(p))
print(id(q))
```


Shallow copy and Deep copy:

Python has a copy module that allows a user to copy an object from one variable to another. As we know that when we assign one variable to another then both the variable just refers to a single object. So, we use the copy module of python to manage the copying process of an object.

There are two types of copy:
1) Shallow Copy [copy.copy()]
2) Deep Copy [copy.deepcopy()]
Let's understand how we can use both of these function one by one.

1) Shallow Copy:

A shallow copy creates a new object but stores reference to the original elements. When we modify one object, the changes do not affect the other object because two different IDs are created for each object.

But when a shallow copy is performed on the nested list then it creates a new object for the outer list, it does not create independent copies of the nested objects.

Consequently, when a modification is made to the nested list, the change is also reflected in other list, as both lists share references to the same nested objects

```
import copy
list1 = [1, 2, 3]
list2 = copy.copy(list1)
list1[1] = 4
print('Old List:', list1)
print('New List:', list2)
```

Output

Old List: [1, 4, 3]

New List: [1, 2, 3]

However, in the case of a nested object, a shallow copy only copies references to the nested objects and does not create separate nested objects. Changes to the nested objects will be reflected in both copies.

```
import copy
list1 = [1, [100,200], 3]
list2 = copy.copy(list1)
print('Old List before:', list1)
list1[1][0] = 4
print('Old List:', list1)
print('New List:', list2)
```

Output

Old List before: [1, [100, 200], 3]

Old List: [1, [4, 200], 3]

New List: [1, [4, 200], 3]

2) Deep Copy:

A deep copy creates a new object and recursively copies all nested objects within the original elements. The copy module provides a method to create a deep copy.

```
import copy
list1 = [1, 2, 3]
list2 = copy.deepcopy(list1)
list1[1] = 4
print('Old List:', list1)
print('New List:', list2)
```

Output

Old List: [1, 4, 3]

New List: [1, 2, 3]

Example: Deep copy of Nested Lists

```
import copy
list1 = [1, [100,200], 3]
list2 = copy.deepcopy(list1)
print('Old List before:', list1)
list1[1][0] = 4
print('Old List:', list1)
print('New List:', list2)
```

Output

Old List before: [1, [100, 200], 3]

Old List: [1, [4, 200], 3]

New List: [1, [100, 200], 3]

Regular expression

Regular Expression(RegEx):

A Regular Expression (RegEx) is a sequence of characters that defines a search pattern.

For example,

^s.....a$

The above code defines a RegEx pattern. The pattern is: any seven letter string starting with **s** and ending with a.

RegEx can be used to check if a string contains the specified search pattern.

Python has a built-in package called re, which can be used to work with Regular Expressions.

```
import re
pattern = '^s.....a$'
test_string = 'sangola'
result = re.match(pattern, test_string)
if result:
      print("Search successful.")
else:
      print("Search unsuccessful.")
```

Output:

Search successful

In above program our pattern is ^s.....a$ which means pattern starts with letter s and ends with a

In between there are five periods(.....) present, means in our pattern in between a and s there can be any 5 letters . This pattern is matched with test string by method **match() .**

There are other several functions defined in the re module to work with RegEx.

Before we explore that, let's learn about regular expressions themselves.

Specify Pattern Using RegEx :

To specify regular expressions, metacharacters are used. In the above example, ^ and $ are metacharacters.

MetaCharacters :

Metacharacters are characters that are interpreted in a special way by a RegEx engine.

Here's a list of metacharacters:

[] . ^ $ * + ? {} () \ |

[] - Square brackets : Square brackets specifies a set of characters we wish to match**.**

Expression	String	Matched?
[abc]	a	1 match
	ac	2 matches
	Hey Jude	No match
	abc de ca	5 matches

Here, [abc] will match if the string we are trying to match contains any of the a, b or c.
We can also specify a range of characters using - inside square brackets.

[a-e] is the same as [abcde].

[1-4] is the same as [1234].

[0-39] is the same as [01239].

We can complement (invert) the character set by using caret ^ symbol at the start of a square-bracket.

[^abc] means any character except a or b or c.

[^0-9] means any non-digit character.

. - Period : A period matches any single character (except newline '\n').

Expression	String	Matched?
. .	a	No match
	ac	1 match
	acd	1 match
	acde	2 matches (contains 4 characters)

^ Caret : The caret symbol ^ is used to check if a string starts with a certain character.

Expression	String	Matched?
^a	a	1 match
	abc	1 match
	bac	No match
^ab	abc	1 match
	acb	No match (starts with a but not followed by b)

$ Dollar : The dollar symbol $ is used to check if a string ends with a certain character.

Expression	String	Matched?
a$	a	1 match
	formula	1 match
	cab	No match

*** Star :** The star symbol * matches zero or more occurrences of the pattern left to it.

Expression	String	Matched?
ma*n	mn	1 match
	man	1 match
	maaan	1 match
	main	No match (a is not followed by n)
	woman	1 match

+ **Plus :** The plus symbol + matches **one or more occurrences** of the pattern left to it.

Expression	String	Matched?
ma+n	mn	No match (no a character)
	man	1 match
	maaan	1 match
	main	No match (a is not followed by n)
	woman	1 match

? **Question Mark :** The question mark symbol ? matches **zero or one occurrence** of the pattern left to it.

Expression	String	Matched?
ma?n	mn	1 match
	man	1 match
	maaan	No match (more than one a character)
	main	No match (a is not followed by n)
	woman	1 match

{} **Braces :**

Consider this code: {n,m}. This means at least n, and at most m repetitions of the pattern left to it.

Expression	String	Matched?
a{2,3}	abc dat	No match
	abc daat	1 match (at daat)
	aabc daaat	2 matches (at aabc and daaat)
	aabc daaaat	2 matches (at aabc and daaaat)

| **Alternation:** Vertical bar | is used for alternation (or operator).

Expression	String	Matched?
a|b	cde	No match
	ade	1 match (match at ade)
	acdbea	3 matches (at acdbea)

Here, a|b match any string that contains either a or b

() Group : Parentheses () is used to group sub-patterns. For example, (a|b|c)xz match any string that matches either a or b or c followed by xz

Expression	String	Matched?
(a|b|c)xz	ab xz	No match
	abxz	1 match (match at abxz)
	axz cabxz	2 matches (at axzbc cabxz)

\ Backslash

- Backlash \ is used to escape various characters including all metacharacters. For example,
- \$a match if a string contains $ followed by a. Here, $ is not interpreted by a RegEx engine in a special way.
- If you are unsure if a character has special meaning or not, you can put \ in front of it. This makes sure the character is not treated in a special way.

All these are the metacharacters.

Now let's understand about special Sequences

Special sequences: Special sequences make commonly used patterns easier to write. Here's a list of special sequences:

\A : Matches if the specified characters are at the start of a string.

Expression	String	Matched?
\Athe	the sun	Match
	In the sun	No match

\b : Matches if the specified characters are at the beginning or end of a word.

Expression	String	Matched?
\bfoo	football	Match
	a football	Match
	Afootball	No match
foo\b	the foo	Match
	the afoo test	Match
	the afootest	No match

\B : Opposite of \b. Matches if the specified characters are **not** at the beginning or end of a word.

Expression	String	Matched?
\Bfoo	football	No match
	a football	No match
	Afootball	Match
foo\B	the foo	No match
	the afoo test	No match
	the afootest	Match

\d : Matches any decimal digit. Equivalent to [0-9]

Expression	String	Matched?
\d	12abc3	3 matches (at 12abc3)
	Python	No match

\D : Matches any non-decimal digit. Equivalent to [^0-9]

Expression	String	Matched?
\D	1ab34"50	3 matches (at 1ab34"50)
	1345	No match

\s : Matches where a string contains any whitespace character. Equivalent to [\t\n\r\f\v].

Expression	String	Matched?
\s	Python RegEx	1 match
	PythonRegEx	No match

\S : Matches where a string contains any non-whitespace character. Equivalent to [^ \t\n\r\f\v].

Expression	String	Matched?
\S	a b	2 matches (at a b)
		No match

\w : Matches any alphanumeric character (digits and alphabets). Equivalent to [a-z A-Z 0-9 _]. By the way, underscore _ is also considered an alphanumeric character.

Expression	String	Matched?
\w	12&": ;c	3 matches (at 12&" : ;c)
	%"> !	No match

\W : Matches any non-alphanumeric character. Equivalent to [^ a-z A-Z 0-9_]

Expression	String	Matched?
\W	1a2%c	1 match (at 1a2%c)
	Python	No match

\Z : Matches if the specified characters are at the end of a string.

Expression	String	Matched?
Python\Z	I like Python	1 match
	I like Python Programming	No match
	Python is fun.	No match

Now we understand the basics of RegEx, let's discuss how to use RegEx in our Python code.

Python RegEx :

Python has a module named **re** to work with regular expressions. To use it, we need to import the module.

```
import re
```

The module defines several functions and constants to work with RegEx.

re.findall() : The re.findall() method returns a list of strings containing all matches.

```
# Program to extract numbers from a string
import re
str = 'hello 12 hi 89. Howdy 34'
pattern = '\d+'
r = re.findall(pattern, str)
print(r)
```

Here is a string stored in str from which we want to find only numbers

For that we used \d+ special sequence

to find one or more occurrence of Numbers from our string.

To find this we used findall() method

Output: ['12', '89', '34']

re.split() : The re.split() method splits the string where there is a match and returns a list of strings where the splits have occurred.

```
import re
str = 'Twelve:12 Eighty nine:89.'
pattern = '\d+'

r = re.split(pattern, str)
print(r)
```

Output: ['Twelve:', ' Eighty nine:', '.']

Note: If the pattern is not found, re.split() returns a list containing the original string.

re.sub() : The method returns a string where matched occurrences are replaced with the content of replace variable.

Syntax :

```
re.sub(pattern, replace, string)
```

```
# Program to remove all whitespaces
import re

# multiline string
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'

# empty string
replace = ' '


new_string = re.sub(pattern, replace, string)
print(new_string)
```

Output:

abc12de23f456

Note : If the pattern is not found, re.sub() returns the original string.

re.subn()

The re.subn() is similar to re.sub() except it returns a tuple of 2 items containing the new string and the number of substitutions made.

```
# Program to remove all whitespaces
import re

# multiline string
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'

# empty string
replace = ' '


new_string = re.subn(pattern, replace, string)
print(new_string)
```

Output :

('abc 12de 23 f45 6', 4)

re.search()

The re.search() method takes two arguments: a pattern and a string. The method looks for the first location where the RegEx pattern produces a match with the string.

If the search is successful, re.search() returns a match object; if not, it returns None.

Syntax

```
match = re.search(pattern, str)
```

```
import re
string = "Python is fun"

# check if 'Python' is at the beginning
match = re.search('\APython', string)

if match:
  print("pattern found inside the string")
else:
  print("pattern not found")
```

Output: pattern found inside the string

Here, match contains a match object.

Match Object

A Match Object is an object containing information about the search and the result.

Note: If there is no match, the value None will be returned, instead of the Match Object.

Example

```
txt = "The rain in Spain"
x = re.search("ai", txt)
print(x) #this will print an object
```

Output : <_sre.SRE_Match object; span=(5, 7), match='ai'>

The Match object has properties and methods used to retrieve information about the search, and the result:

.span() returns a tuple containing the start-, and end positions of the match.

.string returns the string passed into the function

.group() returns the part of the string where there was a match

.start() returns the first index of the substring matched by group.

.end() returns the last index of the substring matched by group.

Example of span()

Print the position (start- and end-position) of the first match occurrence.

The regular expression looks for any words that starts with an upper case "S":

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.span())
```

Output: (12, 17)

Example of string()

Print the string passed into the function:

```
import re
txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.string)
```

Output : The rain in Spain

Example of group()

Print the part of the string where there was a match.

The regular expression looks for any words that starts with an upper case "S":

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.group())
```

Output: Spain

Example of start()

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.start())
```

Output:12

Example of end

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.end())
```

Output:17

Raw string using r prefix

We can create a raw string in Python by prefixing a string literal with r or R. Python raw string treats the backslash character (\) as a literal character. Raw string is useful when a string needs to contain a backslash, such as for a regular expression or Windows directory path, and we don't want it to be treated as an escape character.

```
s = r'Hi\nHello'
print(s)
```

Output: Hi\nHello